Cloud Kitchen Ordering & Delivery Operations

GROUP 7 | Mohd Fahad Khan, Md Saquib Anjum Khan, Gautam Singh, Shivanshu | Team size: 4 | Duration: 7 days

Backend	Frontend	Database	Auth	Deployment
Flask (REST API)	React + Vite	PostgreSQL + SQLAlchemy	JWT (access + refresh)	Docker Compose

1. Problem Statement

Three portals over one API: customers order from a menu, the kitchen fulfils orders from a live queue, and delivery agents pick up and deliver with an OTP handover.

2. Roles & Permissions

Role	What they can do
Customer	Browses the menu, manages a cart, places and tracks orders, rates after delivery.
Kitchen Manager	Works a live order queue: accept, reject, prepare, mark ready, assign agent.
Delivery Agent	Accepts an assignment, marks picked up, completes delivery with an OTP.
Admin	Manages the menu and agents, sees revenue and operational reports.

3. Data Model

- **User** - credentials, role, phone; **Address** - label, line, pincode.
- **MenuItem** - name, category, price, is_veg, is_available, prep minutes.
- **Cart** + **CartItem** - per customer, menu item, quantity.
- **Order** + **OrderItem** - customer, address, items with price snapshot, item total, delivery fee, tax, grand total, state, OTP.
- **DeliveryAssignment** - order, agent, assigned_at, accepted_at, picked_up_at, delivered_at.
- **OrderStatusLog** - order, from state, to state, actor, timestamp.
- **Rating** - order, stars, comment.

4. Functional Requirements

- Menu browsing with category, veg/non-veg and price filters; unavailable items greyed out.
- Cart operations and address selection; the bill is recomputed server-side at placement.
- Kitchen queue that auto-refreshes, with accept / reject and mark-ready actions.
- Assign a ready order to a free agent; the agent accepts, picks up, and delivers with the OTP.
- Customer order tracking screen that polls the current state and shows the timeline.
- Order history, re-order, and rating after delivery.
- Admin dashboard: orders per day, revenue, average preparation time, agent performance.

5. Core Technical Challenge

One strict order state machine shared by three portals. States: Placed, Accepted, Preparing, Ready, Picked Up, Delivered, with Cancelled allowed only before Preparing and Rejected only from Placed. Each transition is restricted to exactly one role - a customer cannot mark an order ready, an agent cannot accept it on the kitchen's behalf - and every transition writes an `OrderStatusLog` row. Delivery completes only when the submitted OTP matches. An agent may hold at most one active assignment, enforced in the database, so two simultaneous assignments cannot both succeed.

6. React Screens

- Customer app: menu, cart, checkout, tracking, history, rating.
- Kitchen console: queue board with timers and action buttons.
- Agent screen: current assignment with the delivery flow and OTP input.
- Admin dashboard with menu management and reports.

7. Authentication & Authorisation Specification

- Passwords hashed with bcrypt - plain text passwords must never touch the database.
- Endpoints: `POST /auth/register`, `/auth/login`, `/auth/refresh`, `/auth/logout`, `GET /auth/me`.
- The access token carries a `role` claim; a custom decorator `@role_required("admin")` guards privileged routes on top of `@jwt_required()`.
- Refresh rotation: a refresh call issues a new access token; logout adds the token JTI to a blocklist so it cannot be reused.
- React side: Axios request interceptor attaches the token, response interceptor catches 401, calls refresh once, retries the original request, and logs the user out if refresh also fails.
- Route protection in React by role - a seeker must never be able to open an admin screen, even by typing the URL.

8. Docker Specification

- `backend/Dockerfile` - Python slim base, requirements installed first for layer caching, served by `gunicorn` (not the Flask dev server).
- `frontend/Dockerfile` - multi-stage: Node stage runs `npm run build`, nginx stage serves the static build and proxies `/api` to the backend.
- `docker-compose.yml` - services `api`, `web`, `db` on one network, a named volume for database data, a healthcheck on `db`, and `depends_on` so the API waits for it.
- All secrets (DB URL, JWT secret key) come from `.env`; `.env.example` is committed, `.env` is not.
- Acceptance test: on a clean machine, `git clone` then `docker compose up --build` gives a working app with seeded demo data.

9. Phase-wise Execution Plan

Phase 1 - Day 1: Foundation

- Finalise the data model and draw the ER diagram / collection diagram before writing code.
- Repo and folder structure: `backend/`, `frontend/`, `docker-compose.yml`, `README.md`.
- Docker skeleton up early - database container running and reachable from the API container on day one.
- User model + full JWT auth flow + role seeding. Do not move on until login and refresh work in Postman.

Phase 2 - Day 2 to 3: Core API

- Every core CRUD endpoint with request validation and clear error responses (400 / 401 / 403 / 404 / 409).
- Role guards on every write route; ownership checks so a user can only touch their own records.
- Pagination, filtering and sorting on all list endpoints - never return the whole table.
- Test every endpoint in Postman and save the collection before you write a single React component.

Phase 3 - Day 3 to 5: React Application

- Routing, auth context, protected route wrapper, login and register screens.
- List, detail and create/edit screens for the main entities with client-side form validation.
- Loading, empty and error states on every screen - no blank page while data is fetching.
- Shared Axios instance; no API URL hardcoded in components.

Phase 4 - Day 5 to 6: Core Business Logic

- Implement the core rule described above - this is what separates this project from a CRUD demo.
- Dashboard and aggregation endpoints computed in the database layer, not in JavaScript.
- Search and filters wired end to end.

Phase 5 - Day 6 to 8: Harden and Ship

- Full dockerization verified from a clean clone; fix anything that only worked on your laptop.
- Seed script that creates demo users for each role plus realistic sample data.
- README: architecture overview, setup steps, environment variables, API endpoint table, demo credentials.
- Postman collection, bug fixing pass, and a 5-minute demo recording walking through each role.

10. Work Split

Owner	Responsibility
Member 1	Auth, roles, menu, addresses, cart.
Member 2	Order placement, bill computation, state machine, status log.
Member 3	Delivery assignment, agent flow, OTP, ratings, admin reports.
Member 4	React - customer app; kitchen and agent screens shared with members 2 and 3.

Every member must commit backend code and review at least one pull request - no single member owns the whole repository.

11. Deliverables

- GitHub repository containing both applications with a readable commit history from every member.
- README.md with setup instructions, environment variables and an API endpoint table.
- ER diagram (SQL) or collection diagram (MongoDB).
- Seed script with demo data and demo credentials per role.
- Postman collection covering every endpoint.
- Working `docker compose up --build` on a clean machine.
- 5-minute demo video.

12. Evaluation

Area	Weight	What is checked
Functionality	30%	All listed features work end to end without manual database edits

Area	Weight	What is checked
API design & JWT	20%	Correct status codes, validation, role guards, refresh and blocklist working
Core business logic	20%	The project-specific core rule is correctly and safely implemented
React frontend	15%	Component structure, state handling, validation, loading and error states
Docker & docs	15%	One-command startup, clean compose file, complete README and seed data

13. Stretch Goals (only after everything above works)

- Live updates via polling replaced with WebSockets.
- Delivery-fee rule based on pincode distance bands.